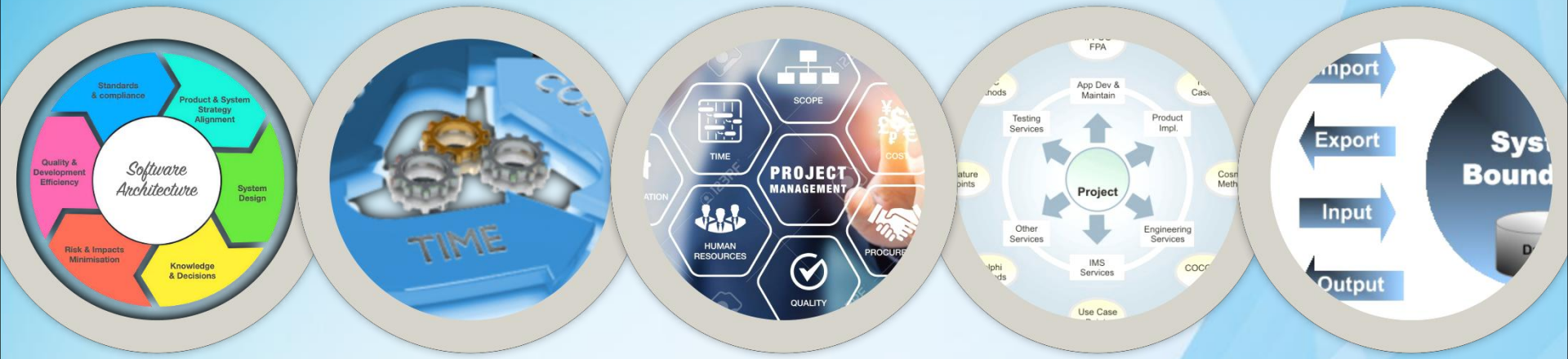


Software Architecture Fundamentals

By: Võ Văn Hải

Email: vovanhaiqn@gmail.com

Chapter 4
Architecture Characteristics Defined

SA - Introduction

- Not just gathering the list of requirement of developing system, the architect must consider many other factors in designing a software solution (figure 4.1).
- Architects may collaborate on defining the domain or business requirements. Still, one key responsibility entails defining, discovering, and otherwise analyzing all the things the software must do that aren't directly related to the domain functionality: architectural characteristics.

*Favor the term **Architecture Characteristics** instead of **nonfunctional requirements** (NFR)*

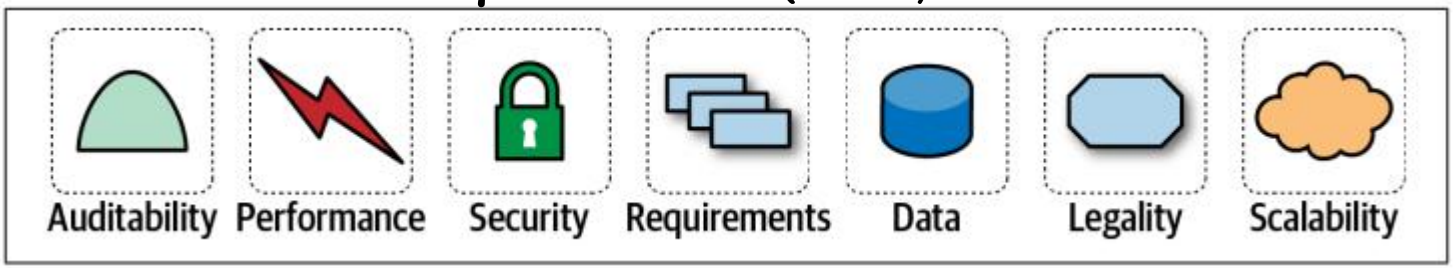


Figure 4-1. A software solution consists of both domain requirements and architectural characteristics

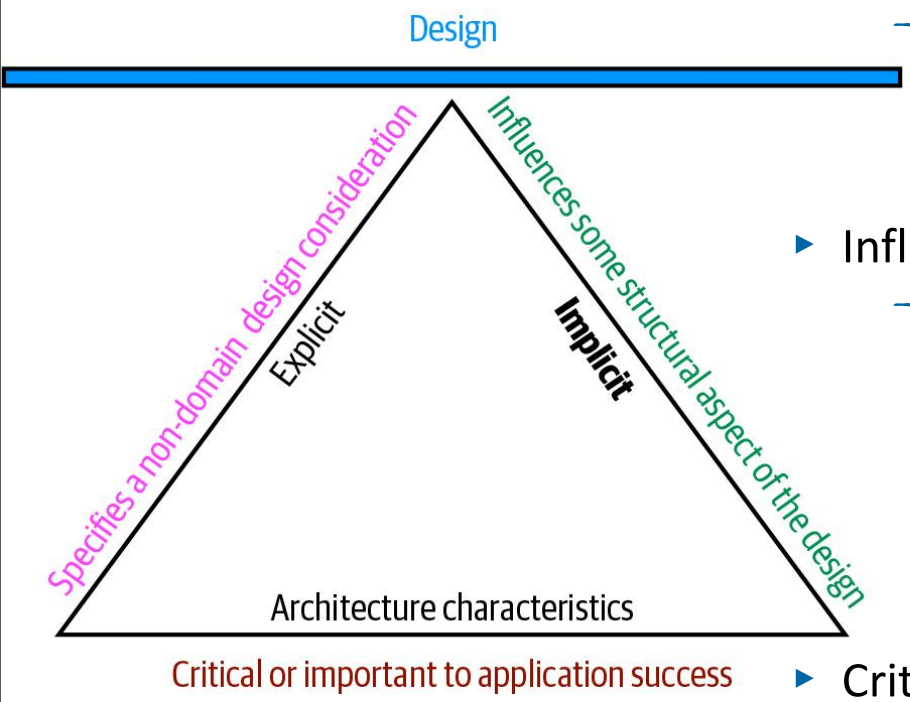
SA - Architecture Characteristics

Architecture characteristic criteria

1. Specifies a non domain design consideration :
   - Requirements specify what the application should do
   - Architecture characteristics specify operational and design criteria for success
   - Concerning how to implement the requirements and why certain choices were made
   - Ex : Level of performance for the application.
2. Influences some structural aspect of the design :
   - Does this architecture characteristic require special structural consideration to succeed
   - Ex : Security aspects
3. Is critical or important to application success :
   - Applications could support a huge number of architecture characteristics…but shouldn't

SA - Architecture Characteristics

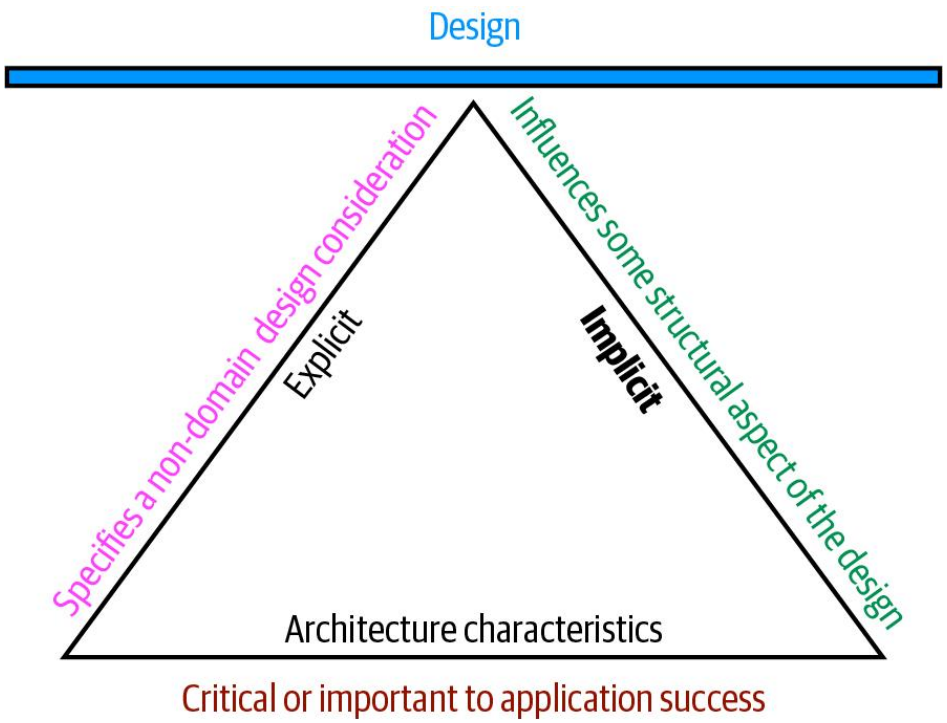
Features of architecture characteristics



- Specifies a nondomain design consideration
  - When designing an application, the requirements specify what the application should do; architecture characteristics specify operational and design criteria for success, concerning how to implement the requirements and why certain choices were made.
- Influences some structural aspect of the design
  - The primary reason architects try to describe architecture characteristics on projects concerns design considerations: does this architecture characteristic require special structural consideration to succeed? For example, security is a concern in virtually every project, and all systems must take a baseline of precautions during design and coding. However, it rises to the level of architecture characteristics when the architect needs to design something special
- Critical or important to application success
  - Applications could support a huge number of architecture characteristics…but shouldn't. Support for each architecture characteristic adds complexity to the design. Thus, a critical job for architects lies in choosing the fewest architecture characteristics rather than the most possible.

SA - Architecture Characteristics

Features of architecture characteristics



- Implicit ones rarely appear in requirements, yet they're necessary for project success.
  - Examples : Availability, reliability, and security
  - Architects must use their knowledge of the problem domain to uncover these architecture characteristics during the analysis phase
- Explicit architecture characteristics appear in requirements documents

**The choice of a triangle is intentional: each definition element supports the others, which in turn helps the system's overall design. The triangle's fulcrum illustrates that these architectural characteristics often interact with one another, leading to the pervasive use of the term trade-off among architects.*

SA - Architectural Characteristics

Operational Architecture Characteristics

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be online again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

SA - Architectural Characteristics

Operational Architecture Characteristics

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

SA - Architectural Characteristics

Cross-Cutting Architecture Characteristics

Term	Definition
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability / Achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.

SA - Architectural Characteristics

ISO 's Architecture Characteristics

The quality mode [ISO 25010 (iso25000.com)]

- No complete list of standards exists. The International Organization for Standards (ISO) publishes a list organized by capabilities, overlapping many of the ones we've listed, but mainly establishing an incomplete category list.
- Read more: pages 81-82
- *** The many ambiguities in Software Architecture (page 83)

SA - Architectural Characteristics – Trade-offs

Trade-Offs and Least-Worst Architecture

- Applications can only support a few of the architecture characteristics for various reasons.
  - First, each supported characteristic requires design effort and perhaps structural support.
  - Second, the bigger problem lies with the fact that each architecture characteristic often has an impact on others.
  - For example, suppose an architect wants to improve security. In that case, it will almost certainly negatively impact performance.
- Thus, architects rarely encounter the situation where they can design a system and maximize every single architecture characteristic. More often, the decisions come down to trade-offs between several competing concerns.
- Too many architecture characteristics lead to generic solutions trying to solve every business problem, and those architectures rarely work because the design becomes unwieldy.

*Never shoot for **the best** architecture, but rather the **least-worst** architecture.*

Chapter 5
Identifying Architectural Characteristics

SA - Identifying Architectural Characteristics

Extracting Architecture Characteristics from Domain Concerns

- An architect must be able to translate domain concerns to identify the right architectural characteristics.
  - For example, is scalability the most important concern, or is it fault tolerance, security, or performance?
  - Perhaps the system requires all four characteristics combined?
- Understanding the key domain goals and domain situation allows an architect to translate those domain concerns to "-ilities," which then forms the basis for correct and justifiable architecture decisions.

Domain concern	Architecture characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Table 5-1. Translation of domain concerns to architecture characteristics

SA - Identifying Architectural Characteristics

Extracting Architecture Characteristics from Requirements

- Some architecture characteristics come from explicit statements in requirements documents.
  - For example, explicit expected numbers of users and scale commonly appear in domain or domain concerns.
- Others come from inherent domain knowledge by architects, one of the many reasons that domain knowledge is always beneficial for architects.
  - For example, suppose an architect designs an application that handles class registration for university students.
  - To make the math easy, assume that the school has 1,000 students and 10 hours for registration. Should an architect design a system assuming consistent scale, making the implicit assumption that the students during the registration process will distribute themselves evenly over time? Or, based on knowledge of university students' habits and proclivities, should the architect design a system that can handle all 1,000 students attempting to register in the last 10 minutes? Anyone who understands how much students stereotypically procrastinate knows the answer to this question! Rarely will details like this appear in requirements documents, yet they do inform the design decisions.

OSA - Case Study: Silicon Sandwiches

Document

- Description
  - A national sandwich shop wants to enable online ordering (in addition to its current call-in service).
- Users
  - Thousands, perhaps one day millions
- Requirements
  - Users will place their order, then be given a time to pick up their sandwich and directions to the shop (which must integrate with several external mapping services that include traffic information)
  - If the shop offers a delivery service, dispatch the driver with the sandwich to the user.
  - Mobile-device accessibility.
  - Offer national daily promotions/specials.
  - Offer local daily promotions/specials.
  - Accept payment online, in person, or upon delivery.
- Additional context
  - Sandwich shops are franchised, each with a different owner.
  - Parent company has near-future plans to expand overseas.
  - Corporate goal is to hire inexpensive labor to maximize profit.

SA - Case Study: Silicon Sandwiches

Explicit Characteristics

- Explicit architecture characteristics appear in a requirements specification as part of the necessary design.
  - Shopping website may aspire to support a particular number of concurrent users, which domain analysts specify in the requirements.
  - An architect should consider each part of the requirements to see if it contributes to an architecture characteristic. But first, an architect should consider domain-level predictions about expected metrics, as represented in the Users section of the kata.
- One of the first details that should catch an architect's eye is the number of users: *currently thousands, perhaps one day millions*.

  → *scalability*—the ability to handle a large number of concurrent users without serious performance degradation—is one of the top architecture characteristics.
- Notice that the problem statement didn't explicitly ask for *scalability*, but rather expressed that requirement as an expected number of users. Architects must often decode domain language into engineering equivalents.

SA - Case Study: Silicon Sandwiches

Explicit Characteristics – sample characteristics

- Scalability: the ability to handle a large number of concurrent users without serious performance degradation.
- Elasticity—the ability to handle bursts of requests.

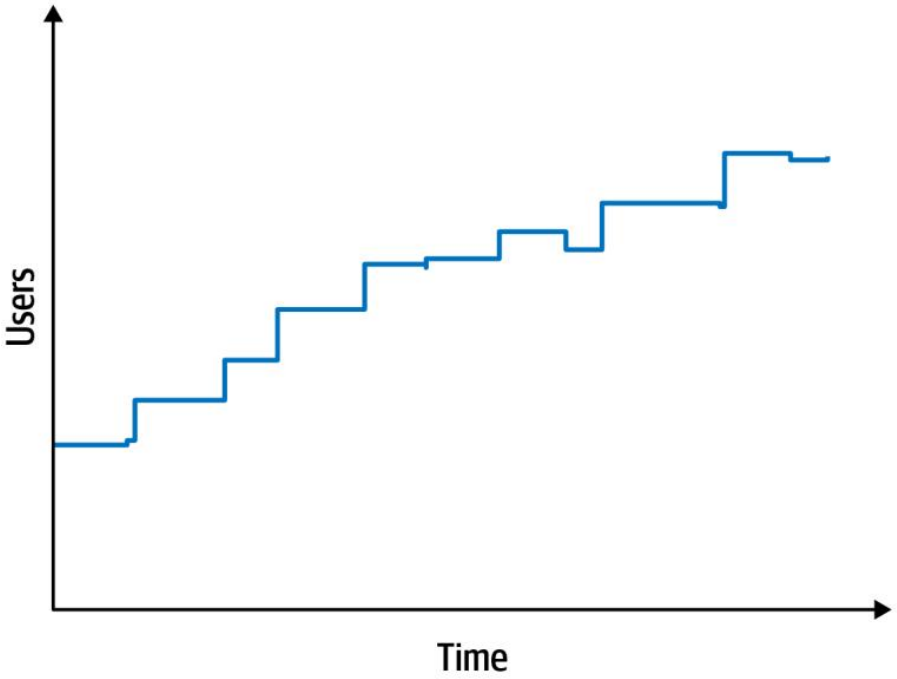


Figure 5-1. Scalability measures the performance of concurrent users

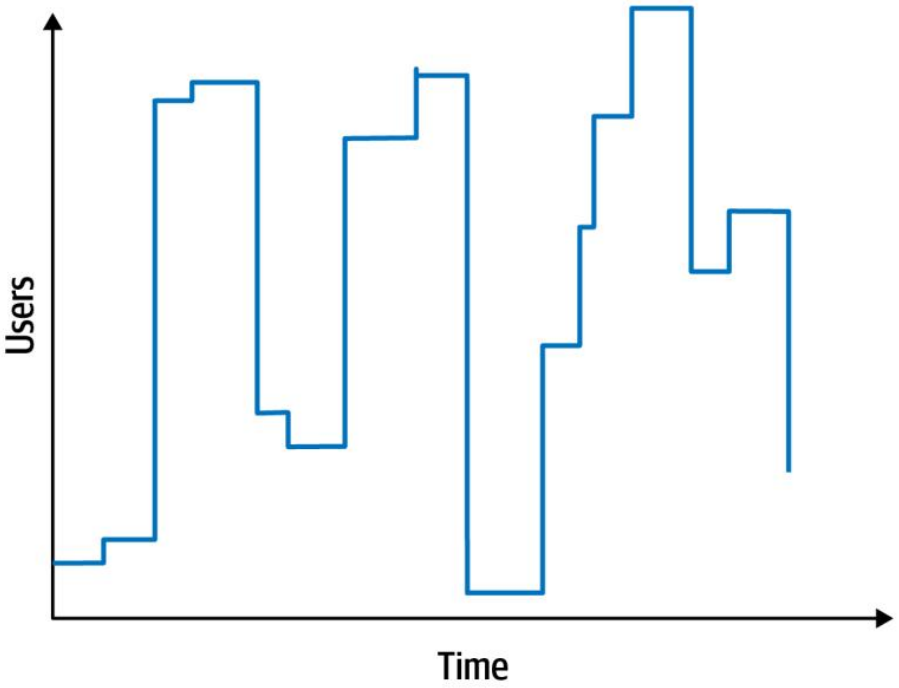


Figure 5-2. Elastic systems must withstand bursts of users

SA - Case Study: Silicon Sandwiches

Implicit Characteristics

- Many architecture characteristics aren't specified in requirements documents, yet they make up an important aspect of the design.
  - One implicit architecture characteristic the system might want to support is ***availability***: making sure users can access the sandwich site.
  - Closely related to availability is ***reliability***: making sure the site stays up during interactions—no one wants to purchase from a site that continues dropping connections, forcing them to log in again.
- ***Security*** appears as an implicit characteristic in every system: no one wants to create insecure software. However, it may be prioritized depending on criticality, which illustrates the interlocking nature of our definition.
  - → An architect considers security an architecture characteristic if it influences some structural aspect of the design and is critical or important to the application.
- …

SA - Case Study: Silicon Sandwiches

Design Versus Architecture and Trade-Offs

- In the Silicon Sandwiches kata, an architect would likely identify customizability as a part of the system, but the question then becomes: architecture or design?
  - The architecture implies some structural component, whereas design resides within the architecture. In the customizability case of Silicon Sandwiches, the architect could choose an architecture style like microkernel and build structural support for customization.
  - However, if the architect chose another style because of competing concerns, developers could implement the customization using the Template Method design pattern, which allows parent classes to define workflow that can be overridden in child classes.

OSA - summary

On what Architecture Characteristics should we focus ?

- *Ineffective architect* : just focus on performance because that seems to be the primary focus of that domain concern.
- Why it's not ?
  - It doesn't matter how fast the system is if it isn't available when needed
  - As the domain grows and more funds are created, the system must be able to also scale to finish end-of-day processing in time
  - The system must not only be available, but must also be reliable so that it doesn't crash as end-of-day fund prices are being calculated
  - What happens if the end-of-day fund pricing is about 85% complete and the system crashes? It must be able to recover and restart where the pricing left off.
  - The system may be fast, but are the fund prices being calculated correctly?
- So, in addition to performance, the architect must also equally place a focus on availability, scalability, reliability, recoverability, and auditability.

Chapter 6
Measuring and Governing Architecture Characteristics

https://www.archunit.org/getting-started

https://github.com/clarkware/jdepend - *https://www.testingtoolsguide.net/tools/jdepend/*

SA - Measuring Architecture Characteristics

Introduction

- Several common problems exist around the definition of architecture characteristics in organizations:
  - They aren't physics
    - Many architecture characteristics in common usage have vague meanings. For example, how does an architect design for agility or deployability? The industry has wildly differing perspectives on common terms, sometimes driven by legitimate differing contexts, and sometimes accidental.
  - Wildly varying definitions
    - Even within the same organization, different departments may disagree on the definition of critical features such as performance. Until developers, architecture, and operations can unify on a common definition, a proper conversation is difficult.
  - Too composite
    - Many desirable architecture characteristics comprise many others at a smaller scale. For example, developers can decompose agility into characteristics such as modularity, deployability, and testability.

SA - Measuring Architecture Characteristics

Operational Measures

- Many architecture characteristics have obvious direct measurements, such as performance or scalability. However, even these offer many nuanced interpretations, depending on the team's goals.
  - For example, perhaps a team measures the average response time for certain requests, a good example of an operational architecture characteristics measure. But if teams only measure the average, what happens if some boundary condition causes 1% of requests to take ten times longer than others? The outliers may not even show up if the site has enough traffic. Therefore, a team may also want to measure the maximum response times to catch outliers.
- High-level teams don't just establish hard performance numbers; they base their definitions on statistical analysis.
  - For example, say a video streaming service wants to monitor scalability. Rather than set an arbitrary number as the goal, engineers measure the scale over time and build statistical models, then raise alarms if the real-time metrics fall outside the prediction models. A failure can mean two things: the model is incorrect (which teams like to know), or something is amiss (which teams also like to know).
- The kinds of characteristics that teams can now measure are evolving rapidly in conjunction with tools and nuanced understanding.
  - For example, many teams recently focused on performance budgets for metrics such as first contentful paint and first CPU idle, both of which speak volumes about performance issues for users of webpages on mobile devices. As devices, targets, capabilities, and myriad other things change, teams will find new things and ways to measure.

SA - Measuring Architecture Characteristics

Structural Measures

- Some objective measures are not so obvious as performance.
  - What about internal structural characteristics, such as well-defined modularity? Unfortunately, comprehensive metrics for internal code quality don't yet exist. However, some metrics and common tools do allow architects to address some critical aspects of code structure, albeit along narrow dimensions.
- Complexity is a prominent measurable aspect of code, defined by the cyclomatic complexity metric.
  - Architects and developers universally agree that overly complex code represents a code smell; it harms virtually every one of the desirable characteristics of code bases: modularity, testability, deployability, and so on.
  - Yet if teams don't keep an eye on gradually growing complexity, that complexity will dominate the code base.

SA - Measuring Architecture Characteristics

Structural Measures - Cyclomatic Complexity

- Cyclomatic Complexity (CC) is a code-level metric designed to provide an object measure for the complexity of code, at the function/method, class, or application level, developed by Thomas McCabe, Sr., in 1976.
- It is computed by applying graph theory to code, specifically decision points, which cause different execution paths.
  - For example, if a function has no decision statements (such as if statements), then CC = 1.
  - If the function had a single conditional, then CC = 2 because two possible execution paths exist.
- The formula for calculating the CC for a single function or method is

$$CC = E - N + 2$$

  - where N represents nodes (lines of code), and E represents edges (possible decisions).
- The number 2 appearing in the cyclomatic complexity formula represents a simplification for a single function/method. For fan-out calls to other methods (known as connected components in graph theory), the more general formula is

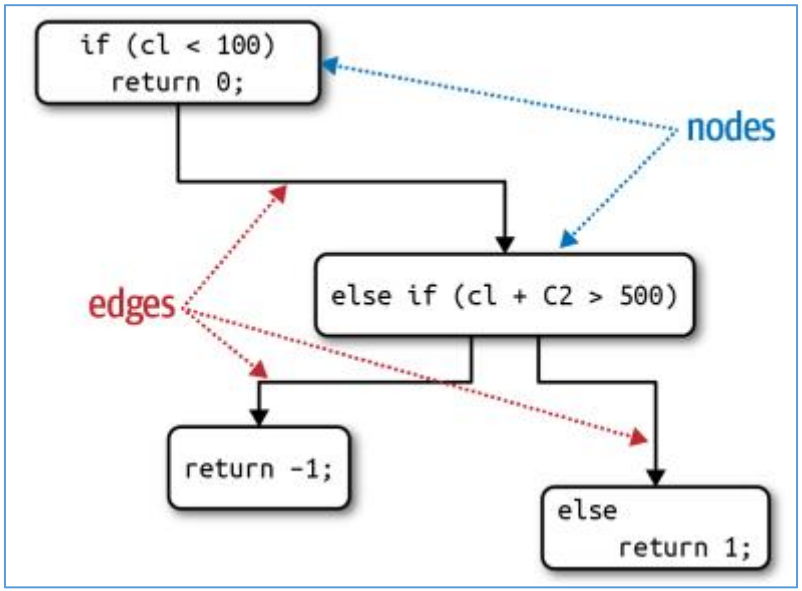
$$CC = E - N + 2P,$$

  - where P represents the number of connected components.

SA - Measuring Architecture Characteristics

Structural Measures - Cyclomatic Complexity (cont.)

```
public void decision(int c1, int c2) {
    if (c1 < 100)
        return 0;
    else if (c1 + C2 > 500)
       return 1;
    else
      return -1;
}
```



$$CC = E - N + 2$$

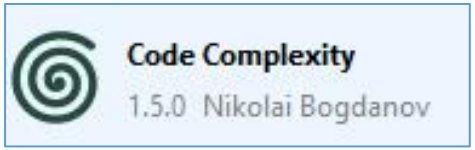
➔CC = 3 – 2 + 2 = 3

- Which is an optimal value for Cyclomatic Complexity?
  - As normal, "It depends!" ☺
  - In general, the industry thresholds for CC suggest that a value under 10 is acceptable, barring other considerations such as complex domains.

SA - Measuring Architecture Characteristics

Structural Measures - Cyclomatic Complexity (cont.)

- Code Complexity is an IntelliJ IDEA plugin for checking the CC of your code in real-time.




```
public class RedBlackTree { very complex (115)
    private Node root;
    private Node TNULL;

    // Preorder
    private void preOrderHelper(Node node) { simple (3)
        if (node != TNULL) {...}
    }
```

```
public class OptionMenu { mildly complex (41)
    Scanner menuInput = new Scanner(System.in);
    DecimalFormat moneyFormat = new DecimalFormat( pattern: "'$'###,##0.00");
    HashMap<Integer, Account> data = new HashMap<~>();

    public void getLogin() throws IOException { mildly complex (11)
        boolean end = false;
        int customerNumber = 0;
        int pinNumber = 0;
        while (!end) {
            try {
```

SA - Measuring Architecture Characteristics

Process Measures

- Some architectural characteristics intersect with software development processes.
  - For example, agility often appears as a desirable feature. However, it is a composite architecture characteristic that architects may decompose into features such as testability and deployability.
- Testability is measurable through code coverage tools for virtually all platforms that assess the completeness of testing. Like all software checks, it cannot replace thinking and intent.
  - For example, a code base can have 100% code coverage yet poor assertions that don't actually provide confidence in code correctness. However, testability is clearly an objectively measurable characteristic. Similarly, teams can measure deployability via a variety of metrics: percentage of successful to failed deployments, how long deployments take, issues/bugs raised by deployments, and a host of others. Each team is responsible for arriving at a good set of measurements that capture useful data for their organization, both in quality and quantity. Many of these measures come down to team priorities and goals.
- Agility and its related parts clearly relate to the software development process. However, that process may impact the structure of the architecture.
  - For example, if ease of deployment and testability are high priorities, then an architect would place more emphasis on good modularity and isolation at the architecture level, an example of an architecture characteristic driving a structural decision. Virtually anything within the scope of a software project may rise to the level of an architecture characteristic if it manages to meet our three criteria, forcing an architect to make design decisions to account for it.

SA - Governance and Fitness Functions

Governing Architecture Characteristics

- The scope of architecture governance covers any aspect of the software development process that architects (including roles like enterprise architects) want to exert an influence upon.
  - For example, ensuring software quality within an organization falls under the heading of architectural governance because it falls within the scope of architecture, and negligence can lead to disastrous quality problems.
- Fortunately, increasingly sophisticated solutions exist from architects to relieve this problem.
  - Extreme Programming created continuous integration, leading to further automation into operations (DevOps)
  - Other agile development methods.

SA - Governance and Fitness Functions

Fitness Functions

- *Architecture fitness function*: *Any mechanism* that provides an objective integrity assessment of some architecture characteristic or combination of architecture characteristics.
  - *any mechanism* — the verification techniques for architecture characteristics are as varied as the characteristics are.
- Fitness functions overlap many existing verification mechanisms, depending on the way they are used: as metrics, monitors, unit testing libraries, chaos engineering, and so on,

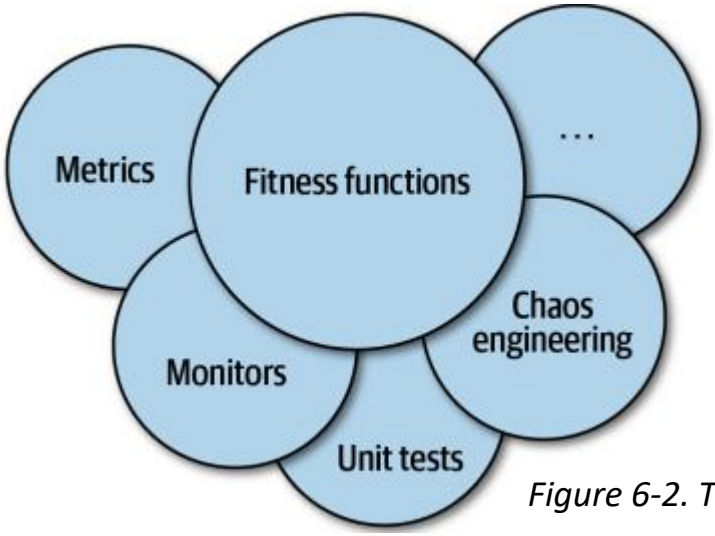
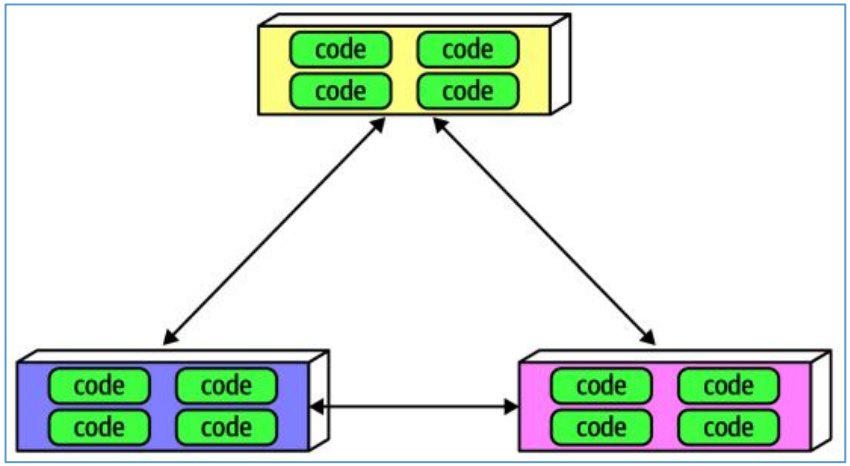


Figure 6-2. The mechanisms of fitness functions

SA - Governance and Fitness Functions

Fitness Functions - Cyclic dependencies

- A Circular Dependency is a relation between two or more modules which either directly or indirectly depend on each other to function properly. Such modules are also known as mutually recursive.
- Having a network of components such as this damages modularity because a developer cannot reuse a single component without also bringing the others along.
- The solution to this problem is to write a fitness function to look after cycles.



SA - Governance and Fitness Functions

Fitness Functions - Cyclic dependencies breaker

Fitness function to detect component cycles

```
public class CycleTest {
    private JDepend jdepend;
    @BeforeEach
    public void init() throws Exception{
        jdepend = new JDepend();
        jdepend.addDirectory("/path/to/project/persistence/classes");
        jdepend.addDirectory("/path/to/project/web/classes");
        jdepend.addDirectory("/path/to/project/thirdpartyjars");
    }
    @Test
    public void testAllPackages() {
        Collection<JavaPackage> packages = jdepend.analyz
        Assertions.assertFalse(jdepend.containsCycles(), "Cyc
    }
}
```

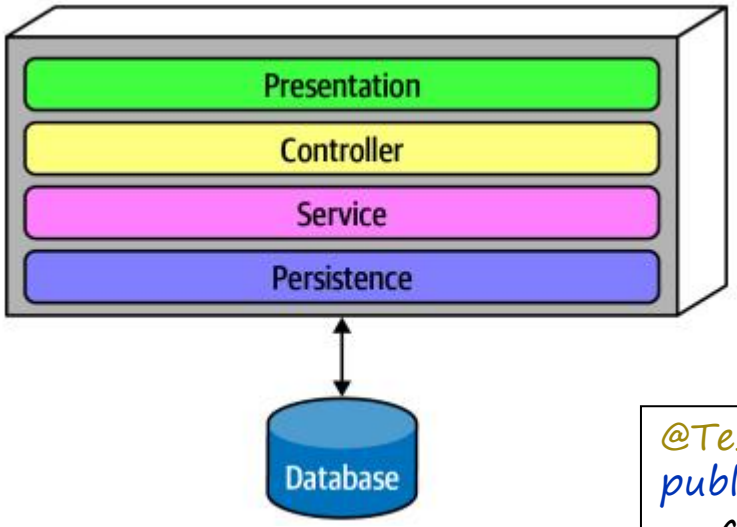
Distance from the main sequence fitness function

```
@Test
void AllPackages() {
    double ideal = 0.0;
    double tolerance = 0.5; // project-dependen
    Collection<JavaPackage> packages = jdepend.
    Iterator<JavaPackage> iter = packages.iterato
    while (iter.hasNext()) {
        JavaPackage p = (JavaPackage) iter.next()
        Assertions.assertEquals(ideal,
                p.distance(),
                tolerance,
                "Distance exceeded: "
                        + p.getName());
    }
}
```

Architects must ensure that developers understand the purpose of the fitness function before imposing it on them.

SA - Governance and Fitness Functions

Fitness Functions - check layers respectation



Layered architecture

ArchUnit, a Java testing framework inspired by and using several parts of the JUnit ecosystem. ArchUnit provides a variety of predefined governance rules codified as unit tests and allows architects to write specific tests that address modularity.

```
@Test
public void checkLayerLeak(){
    com.tngtech.archunit.library.Architectures.
    layeredArchitecture()
            .consideringOnlyDependenciesInLayers()
            .layer("Controller").definedBy("..controller..")
            .layer("Service").definedBy("..service..")
            .layer("Persistence").definedBy("..persistence..")
            .whereLayer("Controller").mayNotBeAccessedByAnyLa
            .whereLayer("Service").mayOnlyBeAccessedByLayers("
            .whereLayer("Persistence").mayOnlyBeAccessedByLaye
}
```

[ArchUnit User Guide]

CHAPTER 7
Scope of Architecture Characteristics

OSA - X

OSA - X

OSA - X

OSA - X

OSA - X

OSA - X

OSA - X